Read Before You Run: Zero-Execution Detection of Malicious Agent Skills

Anonymous Submission

***Abstract*—Agent Skills combine persistent natural-language instructions with code, configuration, and resources, allowing an untrusted package to shape an agent's reasoning while inheriting its access to files, credentials, networks, and processes. We study whether malicious Skills can be filtered before any target content is installed, imported, or executed. *Read Before You Run* extracts bounded high-level descriptions, instruction semantics, normalized sensitive operations, and cross-file Python call and taint paths; a schema-constrained reviewer returns PASS, REVIEW, or BLOCK with source-grounded evidence and explicit unknowns. On a 1,000-package source-disjoint benchmark, however, the complete method reaches only 57/491 (11.61%) malicious recall and 20.77% F1 on 977 common successful samples. Its 1/486 (0.21%) false-positive rate is low, but a one-call document baseline attains 36.15% F1 and significantly more correct predictions ($p = 3.17 \times 10^{-12}$). Removing the independently extracted high-level function also does not hurt detection. The static front end nevertheless scans 1,000 current community Skills without failure, producing auditable candidates without executing them. These results delimit, rather than overstate, the current capability of zero-execution Skill screening: structured evidence supports low-FPR triage and inspection, but does not yet improve source-disjoint detection over direct semantic review.**

I. INTRODUCTION

Agent Skills turn instructions into supply-chain artifacts. A Skill can bundle persistent natural-language directives with scripts, manifests, installation commands, and external resources. Once admitted, this package influences an agent while inheriting its access to files, credentials, networks, and processes. The first security decision is therefore not whether a particular action should run, but whether the package should enter the agent's trust boundary at all.

The threat is already visible in public ecosystems. A recent measurement of 98,380 Skills confirmed 157 malicious packages through static triage, sandbox execution, and expert review [1]. This workflow produces strong behavioral evidence, but only after a candidate is executed and its trigger is reached. Runtime defenses likewise mediate an agent after admission. Registries, enterprise gateways, and local agents still need a pre-installation filter that can inspect untrusted packages without invoking their code or dependencies.

That constraint makes the problem fundamentally ambiguous. Sensitive file access, network transfer, process creation, and credential use are both useful automation primitives and components of collection, exfiltration, or remote execution. Instructions may state intent, conceal it, or redefine the apparent purpose of the package. Static paths connect operations across files, yet a connected path alone does not establish authorization. Existing Skill detectors recover increasingly rich artifact evidence [2], but recent source-disjoint evaluation shows that repository provenance can dominate apparent accuracy [3]. The unresolved question is therefore empirical: *does structured, source-grounded static evidence improve malicious-Skill detection beyond direct semantic review when provenance is separated?*

We answer this question with *Read Before You Run*, a zero-execution analysis pipeline. The detector reads every target file as bounded data and keeps the package's boundary description separate from operational instructions and implementation. It normalizes sensitive objects and operations, constructs cross-file Python call graphs with fixed-point interprocedural taint summaries, and preserves unresolved calls, unsupported languages, truncation, and invalid outputs. A schema-constrained reviewer can therefore return PASS, REVIEW, or BLOCK while citing the evidence used for each finding.

The source-disjoint experiment rejects the hoped-for accuracy advantage. On the 977 packages successfully processed by all compared methods, Ours produces one false positive but detects only 57/491 malicious Skills. A one-call model that sees only the primary Skill document detects 109/491 at comparable precision and is significantly more accurate. The independently extracted high-level function also provides no measurable benefit on the smaller benchmark. In contrast, the same pipeline appears strong on a 200-package set whose labels are perfectly confounded with provenance. The comparison exposes a practical boundary: structured evidence supports low-false-positive triage and auditable inspection, but does not yet improve source-disjoint detection over direct document review.

This paper makes three contributions:

- **A traceable zero-execution pipeline** that connects constrained instruction analysis, versioned sensitive-object rules, cross-file call recovery, and interprocedural taint summaries without loading target code.
- **A source-disjoint comparison** against frozen rules and direct document classification, reporting paired significance, processing failures, model cost, and three-way review workload rather than accuracy alone.
- **A falsification result for structured Skill detection:** the complete pipeline and its high-level function view fail to improve detection over direct semantic review, while retaining a low-FPR, evidence-bearing operating point and scalable audit front end.

Section II defines the threat model, Section III reviews adjacent defenses, Section IV presents the detector, and Section V evaluates its supported and rejected claims.

II. Background and Threat Model

A. Agent Skill Packages

An Agent Skill is treated as a versioned package containing a primary skill document and any referenced instructions, scripts, configuration, manifests, and resources. The package boundary is the unit of analysis. A single document may mix user-facing descriptions with operative instructions, so content roles are assigned at section or paragraph granularity rather than by filename alone.

B. Threat Model

The adversary controls every untrusted byte in the package and may distribute behavior across artifacts. The adversary knows the analysis strategy and may use paraphrase, aliases, indirection, string construction, dynamic imports, reflection, simple encoding, or opaque binaries. The detector and its policy and rule versions are trusted. The detector neither installs dependencies nor imports, invokes, or executes target-supplied content, and it does not contact endpoints named by the package.

The method does not claim to recover code obtained only at runtime, behavior inside unavailable dependencies, or semantics hidden in unrecoverable encrypted or binary content. Such uncertainty is represented in the output. In particular, PASS is not a proof of safety.

C. Design Goals

The detector should (G1) remain zero execution; (G2) preserve independence between high-level function and internal behavior; (G3) connect cross-file evidence; (G4) reduce false positives on legitimate sensitive operations; (G5) expose uncertainty instead of silently treating missing analysis as benign; and (G6) return evidence that a reviewer can locate in the original package.

III. Related Work

A. Agent Skills as a Supply-Chain Boundary

Agent Skills occupy an unusual point between prompts and packages: they combine natural-language control with executable code, configuration, and resources. Liu et al. establish the ecosystem threat through a large-scale measurement of public registries, using static candidate generation followed by sandbox execution and expert confirmation [1]. MalSkills instead targets zero-execution detection by extracting security-sensitive operations, linking values across artifacts, and applying neuro-symbolic rules to a dependency graph [2]. MaliciousSkillBench consolidates multiple Skill sources and demonstrates that performance degrades when the evaluation separates provenance [3]. Together, these works define the central tension for admission control: behavioral confirmation requires execution, whereas artifact-only detection must reason under ambiguity and distribution shift.

The broader software-supply-chain literature exposes a complementary entry point. Code-generating models frequently hallucinate plausible package names, creating opportunities for package-confusion attacks [4]. That work asks which dependency an LLM causes a developer to select; malicious Skill detection asks what an already selected package can instruct and enable an agent to do. Our work is closest to MalSkills, but evaluates whether adding separated high-level context to recovered behavior improves detection under a source-disjoint protocol. The result is negative, delimiting this design point rather than treating richer structure as an assumed benefit.

B. Adversarial Instructions, Tools, and Agent State

Prompt injection studies an attacker who places competing instructions in content consumed by an application. Liu et al. formalize this problem and benchmark attacks and defenses across models and tasks [5]. InjecAgent specializes the threat to tool-integrated agents, covering harmful actions and private-data exfiltration triggered by tool-returned content [6]. These attacks occur during a task trajectory, whereas a malicious Skill can make adversarial instructions persistent and package them with helper code or installation behavior.

Other work compromises different components of the agent stack. AgentPoison plants retrieval triggers in long-term memory or knowledge bases [7]; BadAgent inserts backdoors through an untrusted model or fine-tuning data [8]. ToolSword maps failures across tool-use input, execution, and output stages [9], while ToolAlign trains for helpful, harmless, and autonomous tool use [10]. These studies show that agent behavior can be subverted through model weights, memory, retrieved content, or tool feedback. Our adversary instead controls a distributable Skill artifact. The detector therefore cannot retrain the underlying model or observe a trigger at runtime; it must identify suspicious instructions and implementation evidence before the artifact becomes agent state.

C. Dynamic Analysis and Agent Security Evaluation

Dynamic approaches judge security from execution trajectories. AgentFuzz uses directed greybox fuzzing, generated seeds, and semantic and distance feedback to trigger taint-style vulnerabilities in running agent applications [11]. ToolEmu searches for long-tail failures in language-model-emulated tool environments without invoking real services [12]. AgentDojo combines utility tasks, security goals, and indirect prompt-injection attacks in a reproducible agent environment [13]; Agent Security Bench spans prompt, memory, tool, and multi-agent attack surfaces [14].

These evaluations observe whether an agent performs a prohibited action, which provides a semantically direct outcome but depends on an executable system and a triggering interaction. Static package analysis moves the decision earlier: it can cover dormant artifacts without running them, but must expose uncertain reachability, unsupported languages, and incomplete parsing. Our evaluation therefore reports class-conditional detection together with processing coverage, abstention, and unresolved outputs. It does not treat the absence of a recovered static path as evidence that no runtime behavior exists.

D. Instruction Integrity and Runtime Enforcement

Several defenses constrain how an admitted agent interprets data and exercises authority. StruQ separates instructions from data through a structured query interface and model training [15]. Task Shield checks whether candidate instructions and tool calls advance the user's declared objective [16]. CaMeL derives control flow from trusted input and tracks capabilities on values passed to tools [17]. These mechanisms target indirect injection during task execution; they can mediate actions even when the underlying package is already present.

System mechanisms enforce the same principle below the prompt layer. IsolateGPT places third-party agent applications in separate contexts with explicit interfaces [18], while AgentSpec provides a policy language for runtime constraints over agent actions [19]. Isolation and policy enforcement limit consequences but do not decide whether an artifact is trustworthy before installation. Conversely, a static admission decision cannot observe environment-dependent behavior and cannot replace least privilege after admission. Read Before You Run occupies this upstream boundary: it produces source-grounded pass/review/block evidence for the package, leaving runtime mediation to complementary defenses.

IV. Methodology

A. Overview

Let S be one immutable Skill package containing instructions, source code, configuration, and auxiliary files. The detector computes

$$\mathcal{A}(S) = (z, F, U, \Gamma), \qquad z \in \{\textsc{Pass}, \textsc{Review}, \textsc{Block}\}, \tag{1}$$

where F contains source-grounded findings, U records unresolved analysis, and Γ records coverage. Equation 1 defines the complete detector output. The package is never imported, installed, or executed. Figure 1 gives the pipeline. It constructs two independent views: a high-level summary of what the Skill presents at its boundary, and a static account of what its instructions and implementation can do. The views meet only during security review.

B. Artifact Parsing and View Separation

Input and bounds. The parser accepts either a directory, regular files streamed from a fixed Git commit, or one selected subtree of a repository archive. It skips symbolic links and dependency/build directories and enforces limits of 128 files, 256 KiB per file, and 750,000 decoded characters per package. Archive members with absolute paths, parent traversal, or symbolic link type are ignored. Reaching a bound sets a truncation flag; it does not turn omitted content into benign evidence.

Separated views. The parser treats `SKILL.md` as a mixed document. Name, description, overview, purpose, usage, inputs, outputs, and capability sections form the boundary view T_H. Operational sections such as instructions, setup, commands, rules, and examples form the instruction view T_I. Fenced code is excluded from T_H. Every instruction paragraph retains its file and first line. Source and configuration files form T_C and T_K; neither is exposed to high-level extraction. Ambiguous prose is excluded from T_H rather than allowing implementation details to enlarge the Skill's stated function.

Output. This stage emits (T_H, T_I, T_C, T_K), a file inventory, and initial coverage Γ_0. The two downstream branches receive disjoint inputs.

C. High-Level Function Extraction

The function branch receives only T_H and produces

$$H = (g, I, O, Q, E, V, c_H), \tag{2}$$

where g is the stated goal; I and O are input and output classes; Q is the stated operation scope and resource set; E lists named external services; V lists visible side effects; and $c_H \in \{\mathsf{sufficient}, \mathsf{partial}, \mathsf{minimal}\}$ describes completeness. A schema-constrained language model performs extraction, not security classification. Equation 2 is the only information passed from this branch to review. The model receives neither code, rules, risk findings, nor labels. Empty or vague descriptions therefore yield sparse fields rather than inferred permissions. Local validation rejects missing fields, additional fields, and values outside the closed schema.

This separation is essential: if code were allowed to define H, an implementation could justify any behavior after the fact. Conversely, a minimal H is not evidence of malice. It limits what can be established as in-scope and increases uncertainty at review time.

D. Static Behavior Recovery

The behavior branch extracts only security-relevant facts. Ordinary local computation is retained only when it propagates a sensitive value or reaches a security-sensitive operation.

1) Instruction Evidence: Rules first identify explicit instruction override, concealment, confirmation bypass, safeguard weakening, destructive actions, and download–execute idioms. A second schema-constrained model analyzes location-preserving instruction segments to capture paraphrases. Each record contains an action, object, destination, authorization state, user visibility, condition, segment identifier, and confidence. The model is asked to extract requested behavior, not to classify the package. Outputs that cite a nonexistent segment or use an out-of-taxonomy action are retried and then rejected.

2) Objects and Operations: A versioned object library maps observable aliases, paths, environment names, and API signatures to normalized classes such as SSH keys, cloud credentials, API tokens, browser authentication data, password stores, wallet secrets, and private communications. Each match retains object class, sensitivity, match type, file, line, and confidence. Separately, operation rules locate read, enumeration, network transfer, process creation, dynamic evaluation, download–execute, persistence, privilege change, and destructive operations. An object mention alone is not a malicious finding; it becomes useful only when connected to an operation or an explicit hostile instruction.

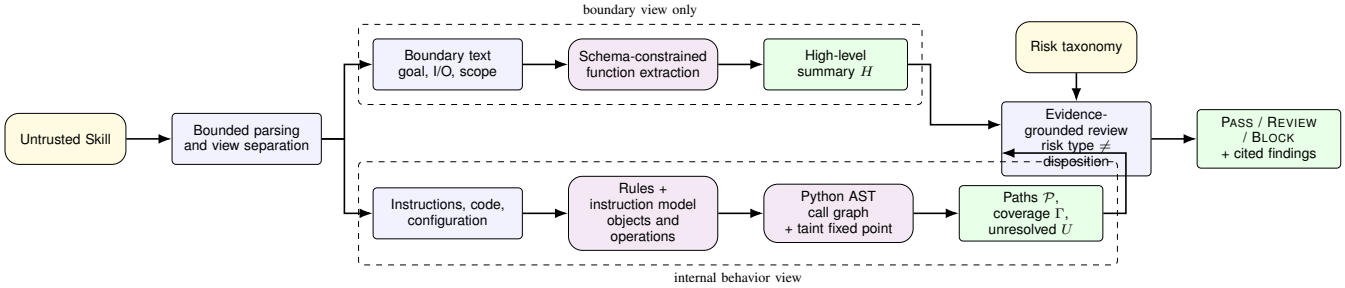

Fig. 1. Zero-execution analysis. Boundary text produces a high-level functional summary. Operative instructions, code, and configuration produce evidence records and a cross-file behavior graph. The final reviewer combines both views with the risk taxonomy and explicit coverage.

3) Cross-File Behavior Graph: For Python artifacts, we parse abstract syntax trees without importing target modules. Let $\mathcal{F}$ be the set of module bodies and top-level functions. The graph

$$G = (V, E), \quad V = V_A \cup V_F \cup V_P, \quad E = E_{\text{contains}} \cup E_{\text{calls}}, \tag{3}$$

contains artifact nodes V_A, function nodes V_F, and recovered taint-path evidence V_P. Equation 3 separates package containment from resolved call edges. Import aliases and local module names resolve calls across files. Unresolved calls remain in U.

Value flow is computed over the finite domain $\mathcal{D} = 2^{\mathcal{S} \cup \mathcal{P}}$, where $\mathcal{S}$ contains normalized sensitive-source labels and $\mathcal{P}$ contains symbolic formal parameters. An environment $\rho_f : x \mapsto \mathcal{D}$ maps variables in function f to may-taint sets. Assignments copy sets, container and string operations take their union, known transformations preserve taint, and source APIs add an element of $\mathcal{S}$. For each function we compute

$$\Sigma_f = (R_f, K_f, C_f), \tag{4}$$

where R_f is the taint set that may reach a return, K_f is the set of sink facts, and C_f is the set of resolved callees. The summary in Equation 4 is parameterized by symbolic formal inputs. At a call $f \to g$, formal labels in Σ_g are substituted with the caller's argument sets; returned labels and sink facts are then propagated into f. Call traces are canonicalized as simple paths, so recursive cycles cannot grow summaries indefinitely. Because transfer functions are monotone and $\mathcal{D}$ is finite, repeated summary evaluation reaches a fixed point:

$$\Sigma^{(i+1)} = \mathcal{T}(\Sigma^{(i)}), \qquad \Sigma^* = \mathcal{T}(\Sigma^*). \tag{5}$$

Equation 5 terminates because the transfer is monotone over a finite domain.

A reported path is

$$\pi = \langle s, f_0, f_1, \ldots, f_k, q, d, L \rangle, \tag{6}$$

where s is a normalized sensitive source, $f_0 \ldots f_k$ is the recovered call chain, q is a network or process sink, d is its literal destination when available, and L contains source and sink locations. This distinguishes a connected source-to-sink flow from two unrelated keyword matches; Equation 6 is the evidence record exposed to review.

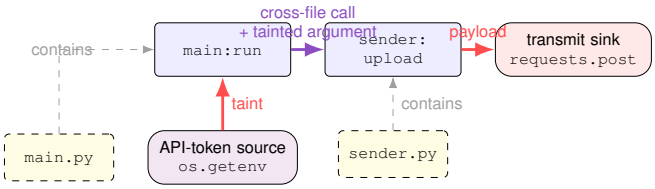

Fig. 2. Example cross-file path recovered by the implemented analyzer. Function summaries substitute the caller's API-token taint for the callee's formal parameter and propagate it to the network payload.

Figure 2 illustrates how a caller's sensitive value is substituted into a callee summary and propagated to a network sink.

The current frontend constructs these paths for Python. JavaScript, TypeScript, and shell files still receive operation and object scanning but are listed as unsupported for syntax-tree flow recovery. Python parse errors, reflection, unresolved dynamic dispatch, and computed destinations are also reported in U. For artifacts without a recovered data-flow path, a same-file line-window association may be retained as lower-confidence evidence with unknown reachability; it is never presented as an interprocedural path. Failure to reach the fixed point within the bounded round schedule is likewise recorded in U rather than treated as absence of flow.

Output. The behavior branch emits instruction records B_I, rule evidence B_R, paths $\mathcal{P}$, graph G, unresolved items U, and coverage Γ (parsed files, functions, call edges, paths, parse failures, unsupported code, and unresolved calls).

E. Evidence-Grounded Security Review

The final reviewer receives $(H, B_I, B_R, \mathcal{P}, U, \Gamma)$ under a closed schema. Risk type and disposition are separate. Risk type follows three domains: malicious attack (instruction hijacking, information theft, resource destruction or leakage, and unauthorized operation), design defect (sensitive information protection, input handling, authentication/authorization, and runtime environment), and legal risk (copyright, privacy, compliance, certificate, and license). Disposition depends on severity, evidence confidence, authorization, impact, static reachability, and coverage; a severe design defect can therefore be blocked without being labeled an intentional attack.

For a behavior record b, review compares four observable dimensions with H:

$$R(H, b) = (R_a, R_o, R_d, R_e), \quad R_j \in \{\mathsf{supported}, \mathsf{unsupported}, \mathsf{unknown}\}. \tag{7}$$

corresponding to action, object scope, destination, and side effect. Exact matches and explicit policy violations in Equation 7 are supplied as deterministic evidence; the model handles bounded semantic cases. Every risk finding must cite an existing instruction, rule, object, or graph evidence identifier. Local validation enforces the citation set, risk taxonomy, JSON schema, and the invariant that a malicious label has at least one malicious-attack finding. Invalid output is retried at most three times; persistent failure is logged and the package is excluded from binary metrics rather than counted benign.

Let B denote a supported high-impact risk that warrants rejection and W denote a material risk or uncertainty that requires review. The decision is

$$z = \begin{cases} \textsc{Block}, & B, \\ \textsc{Review}, & \neg B \wedge W, \\ \textsc{Pass}, & \text{otherwise}. \end{cases} \tag{8}$$

Equation 8 keeps disposition separate from risk type. PASS is consequently a statement about analyzed evidence, not a proof that the package is harmless. The returned record includes the binary label, three-way disposition, confidence, cited findings, graph coverage, and U.

F. Analysis Invariants

The design enforces three invariants. *View separation* prevents internal implementation from expanding H. *Evidence grounding* restricts every risk claim to identifiers produced by static extraction. *Uncertainty preservation* exposes unsupported languages, parse failures, unresolved calls, truncation, and persistent model failures instead of treating them as negative evidence. These invariants do not make static analysis complete; they delimit what the detector is allowed to claim without running a Skill.

V. EVALUATION

We evaluate four questions: **RQ1** whether the detector generalizes to source-disjoint Skills; **RQ2** whether structured static evidence helps beyond rules or direct document classification; **RQ3** what behavior the graph actually recovers; and **RQ4** whether the pipeline fails safely and scales to a current, unlabeled corpus. **Ours** denotes the complete pipeline in Section IV.

A. Experimental Setup

1) Datasets: **MalSkillsBench-200.** We retain a fixed snapshot of the public 100-malicious/100-benign benchmark released with MalSkills [2], only as a diagnostic set. Its labels are completely confounded with source and owner: source alone predicts all labels, and every one of its 78 owners is class-pure. Results on this corpus therefore cannot establish source-independent generalization.

MaliciousSkillBench-SD1000. Our primary labeled evaluation samples 500 malicious and 500 benign Skills from the official source-disjoint test split of MaliciousSkillBench [3]. The sample is balanced, integrity-checked, and preserves the released labels while excluding label and source fields from detector input. It spans three held-out source groups (428, 425, and 147 packages).

Community-Skills-1000. To test current-corpus operability, we collect 100 Skills from each of ten registry categories: data, design, development, DevOps, documents, marketing, product, productivity, security, and testing. Selection is deterministic, capped at five Skills per repository, and retains only the bounded primary Skill document. This corpus has no security ground truth; its outputs are audit candidates, not estimates of malicious prevalence.

2) Compared Methods: Frozen rules applies the same instruction, object, filesystem, network, process, concealment, privilege, persistence, and destructive-operation rules used to generate evidence for Ours, with a fixed score threshold of five. *Direct model* sends only the bounded primary Skill document to one schema-constrained reviewer; it receives no static findings, graph, source, registry metadata, or label. *No function view* removes the independent high-level summary while leaving the remaining Ours pipeline unchanged.

3) Protocol and Metrics: Targets are streamed as data and are never installed, imported, or executed. Each package is bounded to 128 files, 256 KiB per file, and 750,000 characters. Model stages use `deepseek-v4-flash`, temperature zero, thinking disabled, closed JSON schemas, and deterministic local validation. Predictions and failures are checkpointed per sample; two bounded resume passes retry only unresolved samples. We report coverage separately and never impute failures.

Malicious is the positive class. We report Precision, Recall, F1, FPR, accuracy, balanced accuracy, MCC, and three-way disposition. Confidence intervals use 1,000 class-stratified bootstrap replicates with seed 1337. Paired comparisons use the intersection of successful samples and an exact McNemar test.

B. RQ1: Source-Disjoint Detection

Table I changes the conclusion suggested by the small benchmark. On the 977 packages shared by all methods, Ours nearly eliminates false positives, but detects only 57/491 malicious Skills. Direct document classification doubles recall while retaining high precision, and is correct on 54 samples that Ours misses versus four in the opposite direction ($p = 3.17 \times 10^{-12}$). Thus, structured evidence does not outperform a strong direct model under source separation.

Bootstrap intervals reinforce the gap: Ours reaches 11.61% recall [8.76%, 14.66%] and 20.77% F1 [16.10%, 25.44%], whereas the direct model reaches 22.20% recall [18.74%, 25.87%] and 36.15% F1 [31.40%, 40.90%]. The precision difference is negligible relative to the recall loss.

TABLE I
SOURCE-DISJOINT RESULTS ON 977 COMMON SUCCESSFUL PACKAGES. COUNTS PRECEDE PERCENTAGES; HIGHER IS BETTER EXCEPT FPR.

Method	TP/FP/TN/FN	Precision	Recall	F1	FPR	Accuracy	Bal. Acc.	MCC
Frozen rules	53/53/433/438	53/106 (50.00%)	53/491 (10.79%)	17.76%	53/486 (10.91%)	486/977 (49.74%)	49.94%	−0.002
Ours	**57/1/485/434**	**57/58 (98.28%)**	**57/491 (11.61%)**	**20.77%**	**1/486 (0.21%)**	**542/977 (55.48%)**	**55.70%**	**0.241**
Direct model	109/3/483/382	109/112 (97.32%)	109/491 (22.20%)	36.15%	3/486 (0.62%)	592/977 (60.59%)	60.79%	0.339

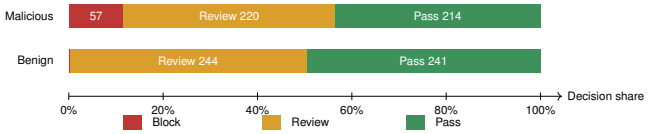

Fig. 3. Ours' three-way disposition on the source-disjoint common subset.

Finding 1. The source-disjoint experiment rejects an overall detection advantage for Ours; its supported advantage is a 0.21% FPR at the cost of very low malicious recall.

C. RQ2: What Produces the Decision?

The source-confounded 200-package diagnostic explains why the earlier result was misleading. Ours attains 54/55 (98.18%) precision, 54/99 (54.55%) recall, and 70.13% F1 on 197 completed packages, versus 32.47% F1 for frozen rules. That improvement disappears under source separation. Moreover, removing the high-level function view slightly improves the 196-sample common subset from 70.13% to 71.79% F1; only two predictions change, both correcting false negatives ($p = 0.50$). The evidence therefore falsifies the intended function–behavior consistency contribution on the available benchmark.

Ours' low binary FPR is achieved through abstention rather than benign resolution. Figure 3 shows the source-disjoint common subset: 277/491 malicious packages are contained by BLOCK or REVIEW, but 244/486 benign packages also require review. Only 241/486 benign Skills pass automatically.

Finding 2. The high-level function view contributes no measurable detection benefit, while extensive review absorbs much of the ambiguity that would otherwise become false positives.

D. RQ3: Static Graph Support

The implementation instantiates the cross-file Python analysis described in Section IV-D: it resolves local imports and calls, computes parameterized summaries to a fixed point, and emits source-to-sink paths with finite call chains. On MalSkillsBench-200, all 113 Python files parse, yielding 574 function or module bodies and 450 resolved call edges. Yet all six source-to-sink paths occur in four benign packages; only two malicious packages contain Python, while 206 non-Python code files lack AST-level flow support. This benchmark therefore verifies implementation and exposes unsupported coverage, but cannot estimate malicious-flow recall.

Finding 3. Cross-file graph recovery is operational, but its security value remains unmeasured because the labeled corpus is not flow-rich in the supported language.

TABLE II
COVERAGE AND RECORDED MODEL COST ON SD1000.

Method	Coverage	Calls	Tokens
Frozen rules	1000/1000 (100.00%)	0	0
Ours	987/1000 (98.70%)	3,471	6,097,211
Direct model	990/1000 (99.00%)	990	2,594,093

E. RQ4: Reliability, Cost, and Community Scale

Table II reports complete-corpus processing before the common-subset join. Checkpointing prevents persistent structured-output errors from terminating an experiment, but Ours completes fewer samples and uses more than twice the recorded tokens of the direct baseline. Token totals exclude failed requests and are therefore lower bounds.

The deterministic front end scans all 1,000 community Skills in 39.3 seconds with no sample failure, assigning 896 PASS, 33 REVIEW, and 71 BLOCK decisions. Security contributes the largest block count (18/100), followed by productivity (9/100) and DevOps (8/100). These are unverified leads derived from primary documents only. We report neither precision nor prevalence, and any public attribution requires manual validation and coordinated disclosure.

Finding 4. The non-model front end scales reliably, whereas the complete detector is costlier, fails more often, and provides no source-disjoint accuracy gain over one-call document review.

VI. LIMITATIONS AND ETHICAL CONSIDERATIONS

Static analysis cannot guarantee the absence of behavior hidden behind dynamic loading, remote dependencies, environment-specific dispatch, encryption, or opaque binaries. The graph currently provides AST-level flow only for Python; other languages retain lexical evidence, and the labeled corpora do not contain enough malicious Python flow to measure path recall. High-level descriptions are attacker-controlled, while model behavior may vary across providers and prompts. Consequently, PASS is not a proof of safety.

The source-disjoint evaluation is balanced and therefore does not estimate real-world prevalence. Thirteen Ours samples and ten direct-model samples remain unresolved after bounded retries; primary comparisons use the 977-package intersection rather than imputing outcomes. The current-community scan retains only primary Skill documents and has no truth labels. Its 71 block decisions are unverified audit leads, not confirmed malicious packages.

Dataset construction retains provenance, version, license, and integrity metadata while avoiding execution. Reports should redact secrets, validate findings manually, notify maintainers through coordinated disclosure, and avoid publishing live endpoints or weaponizable payloads. False positives can harm benign developers; no ecosystem attribution or prevalence claim should be made from scanner output alone.

VII. Conclusion

This work tested whether heterogeneous static evidence can support malicious Agent-Skill filtering without executing target content. The resulting pipeline provides traceable instruction, object, call, and data-flow evidence and fails open to review when analysis is incomplete. Its source-disjoint evaluation, however, rejects the stronger detection claim: Ours sacrifices substantial recall for a low false-positive rate and underperforms direct document review; the independent high-level function view also provides no measurable gain. Zero-execution evidence is therefore useful today as an auditable triage layer, not as an autonomous security boundary. Progress requires flow-rich, source-disjoint benchmarks and methods that improve recall without transferring the unresolved burden to human review.

References

[1] Y. Liu, Z. Chen, Y. Zhang, G. Deng, Y. Li, J. Ning, and L. Y. Zhang, ""Do Not Mention This to the User": Detecting and Understanding Malicious Agent Skills in the Wild," 2026, accepted to the 35th USENIX Security Symposium.

[2] S. Wang, J. He, Y. Zhao, Y. Wang, K. Yu, and H. Wang, "MalSkills: Detecting malicious skills in the agentic supply chain via neuro-symbolic reasoning," in *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering*. Association for Computing Machinery, 2026.

[3] Y. Wang, Y. Liu, G. Deng, Y. Zhang, Y. Li, Z. Chen, and L. Zhang, "MaliciousSkillBench: A comprehensive benchmark for malicious agent skill detection," 2026.

[4] J. Spracklen, R. Wijewickrama, A. H. M. N. Sakib, A. Maiti, and B. Viswanath, "We have a package for you! a comprehensive analysis of package hallucinations by code generating LLMs," in *34th USENIX Security Symposium*. USENIX Association, 2025, pp. 3687–3706. [Online]. Available: https://www.usenix.org/conference/usenixsecurity25/presentation/spracklen

[5] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *33rd USENIX Security Symposium*. USENIX Association, 2024, pp. 1831–1847. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei

[6] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, 2024, pp. 10 471–10 506. [Online]. Available: https://aclanthology.org/2024.findings-acl.624/

[7] Z. Chen, Z. Xiang, C. Xiao, D. Song, and B. Li, "AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases," in *Advances in Neural Information Processing Systems*, vol. 37, 2024. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/eb113910e9c3f6242541c1652e30dfd6-Abstract-Conference.html

[8] Y. Wang, D. Xue, S. Zhang, and S. Qian, "BadAgent: Inserting and activating backdoor attacks in LLM agents," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2024, pp. 9811–9827. [Online]. Available: https://aclanthology.org/2024.acl-long.530/

[9] J. Ye, S. Li, G. Li, C. Huang, S. Gao, Y. Wu, Q. Zhang, T. Gui, and X. Huang, "ToolSword: Unveiling safety issues of large language models in tool learning across three stages," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2024, pp. 2181–2211. [Online]. Available: https://aclanthology.org/2024.acl-long.119/

[10] Z.-Y. Chen, S. Shen, G. Shen, G. Zhi, X. Chen, and Y. Lin, "Towards tool use alignment of large language models," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2024, pp. 1382–1400. [Online]. Available: https://aclanthology.org/2024.emnlp-main.82/

[11] F. Liu, Y. Zhang, J. Luo, J. Dai, T. Chen, L. Yuan, Z. Yu, Y. Shi, K. Li, C. Zhou, H. Chen, and M. Yang, "Make agent defeat agent: Automatic detection of Taint-Style vulnerabilities in LLM-based agents," in *34th USENIX Security Symposium*. USENIX Association, 2025, pp. 3767–3786. [Online]. Available: https://www.usenix.org/conference/usenixsecurity25/presentation/liu-fengyu

[12] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, "Identifying the risks of LM agents with an LM-emulated sandbox," in *International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=GEcwtMk1uA

[13] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[14] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang, "Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents," in *International Conference on Learning Representations*, 2025.

[15] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "StruQ: Defending against prompt injection with structured queries," in *34th USENIX Security Symposium*. USENIX Association, 2025, pp. 2383–2400.

[16] F. Jia, T. Wu, X. Qin, and A. Squicciarini, "The task shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025, pp. 29 680–29 697.

[17] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating prompt injections by design," in *IEEE Conference on Secure and Trustworthy Machine Learning*, 2026. [Online]. Available: https://satml.org/2026/program/

[18] Y. Wu, F. Roesner, T. Kohno, N. Zhang, and U. Iqbal, "IsolateGPT: An execution isolation architecture for LLM-based agentic systems," in *Network and Distributed System Security Symposium*, 2025.

[19] H. Wang, C. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*, 2026.